
FORMALIZING LLM GUARDRAILS WITH AGENTSPEC AND LEAN

Jamie Fulford

Department of Computer Science
University of Illinois Urbana-Champaign
Urbana, IL, USA
jamiehf2@illinois.edu

ABSTRACT

AgentSpec is a domain-specific language for specifying and enforcing runtime constraints on large language model agents. The original work provides definitions, an implementation, and empirical evaluation across multiple domains, but does not formally establish properties about the language’s behavior. We present a mechanized formalization of AgentSpec in Lean 4. Our development provides executable semantics for the core language constructs and machine-checked proofs of key properties. We prove that safety is compositional: safe rule sets can be developed independently and combined without unexpected interactions. We prove that the safety evaluation function correctly characterizes rule violations, with no false positives or misattributions. We prove that each enforcement type satisfies its contract and that the evaluation mechanism terminates. These results establish a formal foundation for AgentSpec and demonstrate that formal methods can be productively applied to LLM agent safety mechanisms.

The code for this project is available at <https://github.com/jfulfo/agentspec-formal>

1 INTRODUCTION

Large language model agents are increasingly deployed in settings where they take autonomous actions. These systems go beyond generating text; they execute code, manipulate files, interact with APIs, and control physical devices. Applications range from software development assistants that write and run code to embodied agents that operate robotic systems to autonomous vehicles that make driving decisions. As these agents are integrated into sensitive workflows, ensuring they operate safely becomes a pressing concern. An agent that executes a harmful command or ignores a traffic signal can cause real damage.

Several approaches have been proposed to mitigate these risks. AgentSpec (Wang et al., 2025) provides a domain-specific language for specifying and enforcing runtime constraints on LLM agents. Users write rules consisting of a triggering event, predicate conditions, and enforcement actions. When an agent attempts an action that triggers a rule and satisfies its conditions, AgentSpec intervenes by stopping execution, requesting user approval, invoking a substitute action, or triggering self-reflection. The original paper demonstrates AgentSpec across multiple domains and shows it can prevent unsafe executions in code agents, eliminate hazardous actions in embodied agents, and enforce traffic law compliance in autonomous vehicles.

While the AgentSpec paper provides definitions for its core concepts and an implementation in Python, it does not establish formal properties about the language. Can safe rule sets be composed safely? Does the safety evaluation correctly identify violations? Does the enforcement mechanism always terminate? Consider a subtle implementation bug where the code evaluating rule violations checked predicates with *or* instead of *and*. Some unsafe actions would slip through despite appearing to satisfy all conditions. Or suppose rule composition silently dropped rules when two programs shared an identifier. Organizations combining independently-developed rule sets might unknowingly

lose protection. These are exactly the kinds of errors that testing struggles to catch, since they require reasoning about all possible inputs rather than a finite sample.

This paper addresses that gap with a mechanized formalization of AgentSpec in Lean 4. We prove that safety is compositional, that the safety evaluation correctly implements the violation semantics, and that each enforcement type satisfies its contract. Our proofs rule out the classes of bugs described above at the semantic level.

2 BACKGROUND

2.1 LLM AGENTS AND GUARDRAILS

Large language model agents extend the capabilities of LLMs by enabling them to take actions in the world. Rather than simply generating text, an agent can execute code, call APIs, send messages, manipulate files, or control physical devices. The agent operates in a loop: it receives a task from the user, reasons about what to do, takes an action, observes the result, and repeats until the task is complete. This execution generates a trajectory of states and actions. While this autonomy makes agents useful for complex tasks, it also introduces risks. An agent might execute a destructive command, leak sensitive information, or take actions that violate organizational policies.

Guardrails are mechanisms that constrain agent behavior to prevent harmful outcomes. A guardrail monitors the agent’s actions and intervenes when certain conditions are met. Interventions might block an action entirely, require human approval before proceeding, or ask the agent to reconsider. The challenge is specifying what conditions should trigger intervention. Different organizations and domains have different requirements, motivating the need for a language that lets users define custom constraints.

2.2 AGENTSPEC

AgentSpec is a domain-specific language for specifying such constraints. A user writes rules, each consisting of a triggering event, a set of predicates that must hold, and a sequence of enforcements to execute when the rule fires. A rule is violated when its event triggers and all its predicates hold; the enforcements then transform the agent’s trajectory accordingly.

2.3 LEAN 4

Lean 4 is a dependently typed programming language and proof assistant. It supports both writing executable programs and constructing machine-checked proofs, making it suitable for formalizing language semantics. We use Lean 4 with the Mathlib library. Our formalization is executable, meaning the semantics we define can be run on concrete inputs, not just reasoned about abstractly.

3 FORMALIZATION

This section presents our Lean 4 formalization of AgentSpec. The goal is an executable semantics that faithfully captures the original paper’s definitions while enabling machine-checked proofs. A recurring tension in formalization work is choosing what to model concretely versus what to leave abstract. We aim to formalize the core language mechanisms while remaining agnostic to domain-specific details like particular predicate implementations.

3.1 CORE TYPES AND TRAJECTORIES

We begin with the fundamental types that model agent execution. A `State` represents the internal state of an LLM agent at a point in time. An `Action` represents a tool invocation or decision the agent can make. An `Observation` captures feedback from the environment after executing an action. These three components combine into a `Step`, which records a single state transition:

```
structure Step where
  fromState : State
  action : Action
```

```

toState : State
observation : Observation

```

A Trajectory consists of an initial state together with a list of steps, representing the complete execution history of an agent. This corresponds to the paper’s notation $\tau = \langle s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} \dots \xrightarrow{a_{n-1}} s_n \rangle$. We define operations `currentState`, `append`, and `slice` (implementing $\tau[: -i]$) on trajectories. These satisfy expected algebraic properties such as $(\tau.\text{slice } i).\text{slice } j = \tau.\text{slice } (i + j)$.

3.2 EVENTS AND PREDICATES

The event system determines when rules are evaluated. We distinguish general lifecycle events (`state_change`, `before_action`, `after_action`, `agent_finish`) from domain-specific events corresponding to particular tool invocations. Event triggering is determined by an `EventContext` that captures the current trajectory, the previous state, the action under consideration, and whether the agent is finishing.

Predicates form the conditions checked when a rule is triggered. We define a predicate language with boolean constants, negation, conjunction, disjunction, named predicates, and parameterized functions. The behavior of named predicates is supplied through a `PredicateTable`, which maps identifiers to predicate functions. This design mirrors the `AgentSpec` implementation, where predicates like `is_destructive_cmd` are Python functions that inspect the current context. By parameterizing over the predicate table, we keep the core semantics independent of any particular domain. This choice also lets us prove properties that hold regardless of what predicates a user defines, which would be impossible if we committed to specific predicate implementations.

3.3 RULES AND PROGRAMS

Following Definition 3.1 in the original paper, we represent an `AgentSpec` rule as a structure with four components: a unique identifier, a triggering event η_r , a list of predicate checks P_r , and a list of enforcements E_r .

```

structure Rule where
  id : ident
  trigger : Event
  checks : List Pred
  enforcements : List Enforcement

```

A rule is well-formed if it has at least one enforcement action. A `Program` is a list of rules, with operations for adding rules and retrieving rules by identifier or by matching event.

3.4 ENFORCEMENT SEMANTICS

Enforcements are interventions taken when a rule is triggered and its conditions are satisfied. We define four possible results: `continue_` (allow action to proceed), `skip` (skip the current action), `stop` (terminate execution), and `selfReflect` (trigger LLM self-examination). The enforcement actions include `stop`, `skip`, `userInspection`, `llmSelfReflect`, and `invokeAction`, each corresponding to a mechanism in Definition 3.3 of the original paper.

Some enforcements involve external effects. User inspection requires human input, and LLM self-reflection requires invoking a language model. We handle this by introducing an `EnforcementEnv` that abstractly provides results of these interactions. This allows us to reason about enforcement behavior under different assumptions about external responses without committing to a particular implementation. For example, we can prove that user rejection always results in a skipped action, parameterized over whatever mechanism actually obtains the user’s decision. When a rule has multiple enforcements, they are applied in sequence until one produces a non-continue result.

Table 1: Correspondence between AgentSpec definitions and our Lean formalization.

Paper	Lean
$\tau = \langle s_0 \xrightarrow{a_0} s_1 \dots \rangle$	Trajectory
$\tau[: -i]$	Trajectory.slice
η_r (trigger)	Rule.trigger : Event
P_r (predicates)	Rule.checks : List Pred
E_r (enforcements)	Rule.enforcements : List Enforcement
$p_r(u, \tau_i) \in \mathbb{B}$	Pred.eval
Rule violation (Def. 3.2)	isRuleViolated
$\text{Eval}(\tau_i, a_i)$	evalSafety

3.5 RULE VIOLATION AND SAFETY EVALUATION

We formalize Definition 3.2 directly. A rule is violated when two conditions hold: the triggering event occurs, and all predicates evaluate to true.

```
def isRuleViolated (cfg : EvalConfig) (r : Rule)
  (eventCtx : EventContext) (predCtx : PredicateContext) : Bool
:= isRuleTriggered r eventCtx && (evalChecks cfg predCtx
  r.checks).passed
```

Predicate checking uses short-circuit evaluation; once a predicate fails, the rest are skipped. The top-level function `evalSafety` determines whether a planned action is safe with respect to a program by searching for any violated rule:

```
def evalSafety (cfg : EvalConfig) (program : Program)
  (t : Trajectory) (plannedAction : Action)
  (userInput : UserInput) : SafetyResult :=
  let violatedRule := program.rules.find? fun r =>
    isRuleViolated cfg r eventCtx predCtx
  match violatedRule with
  | none => SafetyResult.safe
  | some r => SafetyResult.unsafe_ r ...
```

If no rule is violated, the action may proceed. Otherwise, the result identifies the first violated rule. Table 1 summarizes the correspondence between paper definitions and our formalization.

4 VERIFIED PROPERTIES

We now present the key properties verified about our formalization. The original AgentSpec paper provides definitions and an implementation, but does not formally state or prove properties about the language’s behavior. Our theorems fill this gap. All proofs are machine-checked in Lean 4.

4.1 COMPOSITION SAFETY

AgentSpec is designed for deployment across multiple domains. Organizations may develop rule sets independently for different concerns and later combine them. This raises a natural question: if two rule sets are each safe for some context, is their combination safe? And conversely, if a composition is safe, can we conclude anything about the parts?

We define composition as concatenation of rule lists and prove both directions:

Theorem 1 (Safe Composition Implies Safe Parts). *If the composition of two programs is safe for a given trajectory and action, then each program is individually safe:*

$$\begin{aligned} \text{evalSafety}(p_1.\text{compose}(p_2), \tau, a, u) &= \text{safe} \\ \implies \text{evalSafety}(p_1, \tau, a, u) &= \text{safe} \wedge \text{evalSafety}(p_2, \tau, a, u) = \text{safe} \end{aligned}$$

Theorem 2 (Safe Parts Compose Safely). *If two programs are individually safe for a given trajectory and action, their composition is also safe:*

$$\begin{aligned} evalSafety(p_1, \tau, a, u) = safe \wedge evalSafety(p_2, \tau, a, u) = safe \\ \implies evalSafety(p_1.compose(p_2), \tau, a, u) = safe \end{aligned}$$

Together these establish that safety is compositional. Rule sets can be developed in isolation, then combined with confidence that no unexpected interactions arise. The proofs rely on the structure of `evalSafety`, which searches for violated rules in the program’s list. Since list concatenation preserves membership in both directions, violations in the composition correspond exactly to violations in one of the parts. A related result establishes that adding a non-violated rule to a safe program preserves safety.

4.2 SAFETY CHARACTERIZATION

The original paper defines rule violation (Definition 3.2) but does not formally connect this definition to the safety evaluation function. We prove that `evalSafety` correctly implements the intended semantics:

Theorem 3 (No Violations Implies Safe). *If no rule in a program is violated for the given context, then safety evaluation returns safe:*

$$(\forall r \in p.rules. \neg isRuleViolated(r, \tau, a)) \implies evalSafety(p, \tau, a, u) = safe$$

Theorem 4 (Unsafe Identifies Violator). *If safety evaluation returns unsafe citing rule r , then r is in the program and is violated:*

$$evalSafety(p, \tau, a, u) = unsafe(r) \implies r \in p.rules \wedge isRuleViolated(r, \tau, a)$$

These theorems rule out two failure modes. Theorem 3 ensures there are no false positives: if every rule’s conditions fail to hold, the system will not block the action. Theorem 4 ensures there are no false negatives or misattributions: if the system reports a violation, it has correctly identified a rule that actually fired.

4.3 ENFORCEMENT AND TERMINATION GUARANTEES

The enforcement mechanism is where `AgentSpec` intervenes in agent execution. We verify that each enforcement type satisfies its intended contract. The `stop` enforcement must terminate the agent; we prove it always produces a stop result with a finish action. User inspection must respect the user’s decision; we prove that if the user rejects, the enforcement result is skip. LLM self-reflection must respect configured bounds; we prove that when reflection depth reaches the maximum, self-reflection produces skip rather than continuing indefinitely.

We prove that program evaluation always terminates. This property ensures enforcement cannot cause the agent to hang during safety checking. We verify additional supporting properties: predicate evaluation is extensional with respect to predicate tables, satisfies De Morgan’s laws and short-circuit semantics, and conjunction of a predicate list holds exactly when every predicate in the list holds.

5 DISCUSSION

Our formalization covers the core semantics of `AgentSpec`: the structure of rules, the event and predicate system, the enforcement mechanism, and the safety evaluation function. Several aspects lie outside our scope. We do not formalize the parsing layer that transforms `AgentSpec` source text into rule structures. We also do not formalize specific domain predicates; these are treated as abstract functions supplied via the predicate table.

A key design decision was abstracting external effects through the `EnforcementEnv` structure. User inspection and LLM self-reflection involve interactions that cannot be modeled purely within the semantics. By parameterizing over the environment, we can state and prove properties like “if the user rejects, the action is skipped” without committing to how user input is obtained. Our Lean

formalization and the AgentSpec Python implementation are separate artifacts; bugs in the Python implementation are not ruled out by our proofs. Verified code extraction or a reference interpreter checked against our semantics would address this gap, but is beyond the scope of this work.

6 RELATED WORK

LLM Guardrails. Several systems aim to constrain LLM behavior at runtime. NeMo Guardrails (Reborea et al., 2023) provides a toolkit for adding programmable constraints to LLM-based conversational systems. It uses a runtime inspired by dialogue management to enforce user-defined rails that are independent of the underlying model. Llama Guard (Inan et al., 2023) takes a different approach, fine-tuning an LLM to classify inputs and outputs according to a safety taxonomy. Guardrails AI (Rajpal, 2023) provides validators that check LLM outputs against schemas and policies. These systems focus on conversational safety or output validation. AgentSpec differs in targeting *agentic* behavior, where the concern is not just what the model says but what actions it takes in the world. Our formalization provides formal guarantees about AgentSpec’s rule evaluation that these systems lack.

Constrained Generation. Another line of work constrains LLM outputs syntactically. SynCode (Ugare et al., 2025a) uses grammar-augmented decoding to ensure outputs conform to a context-free grammar, eliminating syntax errors in generated code. IterGen (Ugare et al., 2025b) extends this with backtracking, allowing corrections during generation. These approaches enforce syntactic well-formedness of individual outputs rather than safety properties over action sequences. They are complementary to AgentSpec: one might use SynCode to ensure an agent’s generated code is syntactically valid while using AgentSpec to ensure the agent does not execute destructive commands.

DSL Formalization. Formalizing domain-specific languages in proof assistants is established practice. Our contribution is applying this methodology to a language designed for LLM agent safety. The unpredictable nature of LLM behavior makes formal guarantees particularly valuable; we cannot rely on the agent behaving consistently, but we can ensure the guardrail mechanism itself is correct.

7 CONCLUSION

We have presented a mechanized formalization of AgentSpec, a domain-specific language for runtime enforcement of LLM agent behavior. Our Lean 4 development provides executable semantics for the core language constructs and machine-checked proofs of key properties. The composition theorems establish that rule sets can be developed modularly and combined safely. The safety characterization theorems confirm that the evaluation function correctly implements the paper’s definition of rule violation. The enforcement guarantees verify that each intervention type behaves according to its specification.

Several directions remain for future work. Connecting the Lean formalization to the Python implementation through code extraction or verified interpretation would close the gap between verified semantics and deployed system. Extending the formalization to cover temporal properties across multiple agent steps could enable reasoning about long-horizon safety.

REFERENCES

- Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabza. Llama guard: Llm-based input-output safeguard for human-ai conversations, 2023. URL <https://arxiv.org/abs/2312.06674>.
- S Rajpal. Guardrails ai, 2023.
- Traian Reborea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails, 2023. URL <https://arxiv.org/abs/2310.10501>.

-
- Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. Syncode: LLM generation with grammar augmentation. *Transactions on Machine Learning Research*, 2025a. ISSN 2835-8856. URL <https://openreview.net/forum?id=HiUZtgAPoH>.
- Shubham Dipak Ugare, Rohan Gumaste, Tarun Suresh, Gagandeep Singh, and Sasa Misailovic. Itergen: Iterative semantic-aware structured llm generation with backtracking. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (eds.), *International Conference on Representation Learning*, volume 2025, pp. 24369–24394, 2025b. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/3d3a9e085540c65dd3e5731361f9320e-Paper-Conference.pdf.
- Haoyu Wang, Christopher M. Poskitt, and Jun Sun. Agentspec: Customizable runtime enforcement for safe and reliable llm agents, 2025. URL <https://arxiv.org/abs/2503.18666>.